

The initial d3 examples used the `enter().append( ... )` approach to adding svg (or html) elements to the DOM, for each data element.

For instance you could create a list of svg text elements using a call like this:

```
let svg = d3.select('#letters')
  .attr('width', 50)
  .attr('height', 500);

let letters = svg.selectAll("text")
  .data(randomLetters()) // create a random list of letters
  .enter()
  .append("text")
  .attr("x", 25)
  .attr("y", (d, i) => i * 20)
  .text(d => d);

//this is a call to create a random list of letters in alphabetical order
const characters = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';
function randomLetters(){
  let charsToReturn = '';

  for(i = 0; i < characters.length; i++){
    random = Math.random() <= .5;
    if(random)
      charsToReturn = charsToReturn+characters[i];
  }

  console.log(charsToReturn);
  return charsToReturn;
}
```

This works if your svg text elements do not change. Here's one example output:

B
C
J
K
L
N
Q
U
V
W
Y

What if you add a button to the page, and when this button is pressed you regenerate a new list of letters. There are currently 9 svg text elements within the DOM. What if when you generate a new list that is 15 characters long, and consisting in a new set of letters. 9 of the SVG text elements on the display need to change to show the new content, and 6 elements need to be added. Conversely, suppose you get a list that is 5 characters long. You need to remove 4 svg text elements and update the remaining 5.

You can accomplish ALL of these changes, using join instead of enter().append(...).

Here is a new version. It will look very similar to the version above, but will use join instead.

```
let svg = d3.select('#letters')

    .attr('width', 50)
    .attr('height', 500);

let letters = svg.selectAll("text")
    .data(randomLetters()) //create a random list of letters
    .join("text")
    .attr("x", 25)
    .attr("y", (d, i) => i * 20+20)
    .text(d => d);

// create a random list of letters in alphabetical order
const characters = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';
function randomLetters(){
    let charsToReturn = '';

    for(i = 0; i < characters.length; i++){
        random = Math.random() <= .5;
```

```

        if(random)
            charsToReturn = charsToReturn+characters[i];
    }

    console.log(charsToReturn);
    return charsToReturn;
}

```

This version works in exactly the same way as the previous version. We make our list of letters once, and append svg text elements once to the page. Now, let's change this code, so now we have a button that updates the list every time we click it.

```

console.log("hello");
let svg = d3.select('#letters')
    .attr('width', 50)
    .attr('height', 500);

updateVis();

function updateVis(){
    console.log("update");
    let letters = svg.selectAll("text")
        .data(randomLetters())
        .join("text")
        .attr("x", 25)
        .attr("y", (d, i) => i * 20+20)
        .text(d => d);
}

function randomLetters(){
    let charsToReturn = '';

    for(i = 0; i < characters.length; i++){
        random = Math.random() <= .5;
        if(random)
            charsToReturn = charsToReturn+characters[i];
    }

    console.log(charsToReturn);
    return charsToReturn;
}

```

Suppose we add to the html page a button as follows:

```
<!DOCTYPE html>
<html>
<head>
  <title>D3 Project Starter Template</title>
  <meta charset="UTF-8">
  <link rel="icon" href="data:;base64,iVBORwOKGO=" />
  <link rel="stylesheet" href="css/style.css">
</head>

<body>
  <!-- Add HTML content, SVG drawing area, ... -->
  <button onclick="setTimeout(updateVis, 200)">Try it</button>
  <svg id="letters"></svg>

  <!-- D3 bundle -->
  <script src="js/d3.v6.min.js"></script>

  <!-- Our JS code -->
  <script src="js/main.js"></script>

</body>
</html>
```

When the button is clicked, we will get a new list of elements. The svg's will be updated to show the new list, and new svg text elements added or removed as the length changes.

Join internally calls:

1. enter- which is the function that creates the new elements.
2. update- which is the function that updates existing elements with new data- and
3. exit - which removes elements that are no longer needed

You can, within the join function, grab the elements that are being added, updated or removed, to do something with them. Here's an example:

```
const characters = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ';

console.log("hello");
let svg = d3.select('#letters')
```

```

        .attr('width', 50)
        .attr('height', 500);

updateVis();

function updateVis(){

let letters = svg.selectAll("text")
    .data(randomLetters())
    .join(
        function(enter) {
            return enter
                .append('text')
                .style('opacity', 1.0);
        },
        function(update) {
            return update
                .style('opacity', .5);
        },
        function(exit) {
            return exit.remove();
        }
    )
    .attr("x", 25)
    .attr("y", (d, i) => i * 20+20)
    .text(d => d);
}

function randomLetters(){
    let charsToReturn = '';

    for(i = 0; i < characters.length; i++){
        random = Math.random() <= .5;
        if(random)
            charsToReturn = charsToReturn+characters[i];
    }

    console.log(charsToReturn);
    return charsToReturn;
}

```

This will style the svg elements that were already present differently than the new elements being added to the DOM.

Download the full example here and play around with the options.

[Example](#)